

Linear Programming 2

Contents

- Review
- Simplex Method Demonstration
- Problem Based Approach (Python)
- Diet Problem (Tutorial Question 5)
- Formulating with Summation Notation
- Manufacturing Problem (Question 4 Adapted)
- Introduction to:
 - Integer and Mixed-Integer Based Programming

Linear Programming Model

A linear programming model can be written as follows

maximise

$$P = 5x + 7y$$

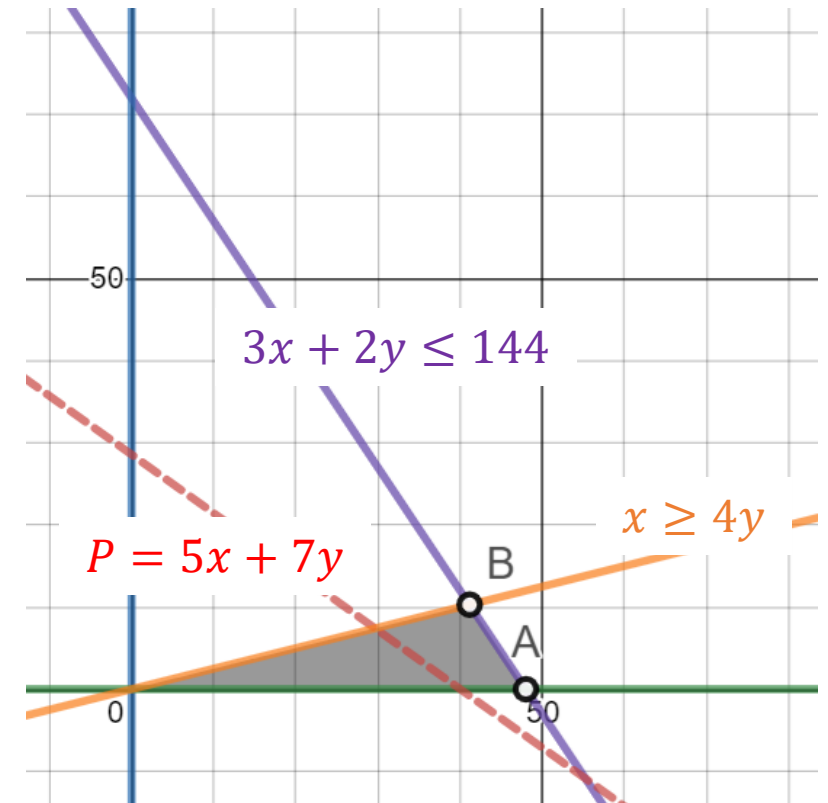
subject to:

$$3x + 2y \leq 144 \quad (\text{C1})$$

$$x \geq 4y \quad (\text{C2})$$

$$x \geq 0$$

$$y \geq 0$$



Canonical Form (Matrices)

Our linear programming model

maximise $5x + 7y$

subject to:

$$3x + 2y \leq 144$$

$$x \geq 4y$$

$$x \geq 0$$

$$y \geq 0$$

maximise $5x + 7y$

subject to:

$$3x + 2y \leq 144$$

$$-x + 4y \leq 0$$

and

$$x \geq 0$$

$$y \geq 0$$

maximise $[5 \ 7] \begin{bmatrix} x \\ y \end{bmatrix}$

subject to:

$$\begin{bmatrix} 3 & 2 \\ -1 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \leq \begin{bmatrix} 144 \\ 0 \end{bmatrix}$$

and

$$\begin{bmatrix} x \\ y \end{bmatrix} \geq \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solution with Python

maximise $\begin{bmatrix} 5 & 7 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$

subject to:

$$\begin{bmatrix} 3 & 2 \\ -1 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \leq \begin{bmatrix} 144 \\ 0 \end{bmatrix}$$

and

$$\begin{bmatrix} x \\ y \end{bmatrix} \geq \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

We are maximising.

The Scipy function `linprog` minimises!

Notice that in the Python code we call `linprog` using `-f`

```
1  from scipy.optimize import linprog
2
3  # cost function (row vector) - to minimise we need the negatives
4  c = [-5, -7]
5
6  # linear inequalities
7  # (no. rows in A = no. rows in b)
8  A_ub = [[3, 2],
9          [-1, 4]]
10 b_ub = [144, 0]
11
12 # call the linprog function
13 results = linprog(c, A_ub, b_ub)
14
15 print(results.message)
16 # if results are a success then print out the optimal solution
17 if results.success:
18     for k in range(len(results.x)):
19         print(f"x[{k+1}] = {results.x[k]:.2f}")
20     print(f"fval = {results.fun:.2f}")
```

Solution with Pyomo (Modelling Language)

maximise $5x + 7y$

subject to:

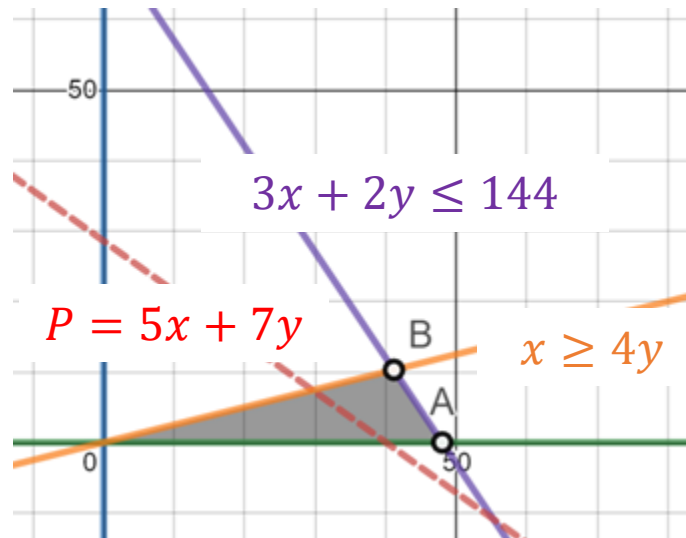
$$3x + 2y \leq 144$$

$$x \geq 4y$$

and

$$x \geq 0$$

$$y \geq 0$$



```
1 from pyomo.environ import *
2
3 # Create a concrete model
4 model = ConcreteModel()
5
6 # Decision variables (non-negative)
7 model.x = Var([0, 1], domain=NonNegativeReals)
8
9 # Objective function (maximize 5x0 + 7x1)
10 f = [5, 7]
11 model.obj = Objective(expr=sum(f[i] * model.x[i] for i in [0, 1]), sense=maximize)
12
13 # Constraints
14 model.c1 = Constraint(expr=3*model.x[0] + 2*model.x[1] <= 144)
15 model.c2 = Constraint(expr=model.x[0] >= 4*model.x[1])
16
17 # Print model before solving
18 print("Model formulation:")
19 model.pprint()
20
21 # Solve with GLPK
22 solver = SolverFactory('glpk')
23 results = solver.solve(model, tee=False)
24
25 # Check results
26 if (results.solver.status == SolverStatus.ok) and \
27     (results.solver.termination_condition == TerminationCondition.optimal):
28     print("\nOptimal solution found:")
29     model.display()
30 else:
31     print("\nSolver did not find an optimal solution. Status:", results.solver.status)
```

Diet Problem (Tutorial Question 5a)

Food	Vitamin A	Vitamin B	Vitamin C	Vitamin D	Fat	Cost	Attractiveness
1	18	4	-	6	-	10	6
2	8	6	22	14	-	6	1
3	22	10	-	-	8	8	5
4	6	12	-	6	-	16	5
5	-	14	-	10	-	8	6
6	-	-	20	8	6	3	4
7	8	-	-	6	20	6	10
8	4	-	8	-	40	8	10
9	-	6	10	12	-	12	6
10	12	20	-	4	-	12	7

	Minimum	Maximum
Vitamin A	100	-
Vitamin B	120	-
Vitamin C	200	-
Vitamin D	80	-
Fat	-	300

The meals provided must satisfy certain basic restriction on vitamin and fat content.
The right table gives the target quantities for a sample of 4 vitamins and one fat type.

The quantities of each nutrient contained in one unit of each of 10 foods and the cost and attractiveness of a unit of each food is given in the left table.

Obtain the optimal solution given that the aim is to minimise the cost of the diet.

Diet Problem (Tutorial Question 5a)

Food	Vitamin A	Vitamin B	Vitamin C	Vitamin D	Fat	Cost	Attractiveness
1	18	4	-	6	-	10	6
2	8	6	22	14	-	6	1
3	22	10	-	-	8	8	5
4	6	12	-	6	-	16	5
5	-	14	-	10	-	8	6
6	-	-	20	8	6	3	4
7	8	-	-	6	20	6	10
8	4	-	8	-	40	8	10
9	-	6	10	12	-	12	6
10	12	20	-	4	-	12	7

	Minimum	Maximum
Vitamin A	100	-
Vitamin B	120	-
Vitamin C	200	-
Vitamin D	80	-
Fat	-	300

Variables

x_i - Number of food i

Objective:

minimise $10x_1 + 6x_2 + 8x_3 + 16x_4 + 8x_5 + 3x_6 + 6x_7 + 8x_8 + 12x_9 + 12x_{10}$

Vitamin A Constraint:

subject to:

$$18x_1 + 8x_2 + 22x_3 + 6x_4 + 8x_7 + 4x_8 + 12x_{10} \geq 100$$

Diet Problem (Tutorial Question 5a)

minimise $10x_1 + 6x_2 + 8x_3 + 16x_4 + 8x_5 + 3x_6 + 6x_7 + 8x_8 + 12x_9 + 12x_{10}$

subject to:

$$18x_1 + 8x_2 + 22x_3 + 6x_4 + 8x_7 + 4x_8 + 12x_{10} \geq 100 \quad (\text{Vitamin A Constraint})$$

$$4x_1 + 6x_2 + 10x_3 + 12x_4 + 12x_5 + 6x_9 + 20x_{10} \geq 120 \quad (\text{Vitamin B Constraint})$$

$$22x_2 + 20x_6 + 8x_8 + 10x_{10} \geq 200 \quad (\text{Vitamin C Constraint})$$

$$6x_1 + 14x_2 + 6x_4 + 10x_5 + 8x_6 + 6x_7 + 12x_9 + 4x_{10} \geq 80 \quad (\text{Vitamin D Constraint})$$

$$8x_3 + 6x_6 + 20x_7 + 40x_8 \leq 300 \quad (\text{Fat Constraint})$$

and

$$x_1, x_2, \dots, x_{10} \geq 0 \quad (\text{Non-negativity})$$

Diet Problem (Tutorial Question 5a)

Food	Vitamin A	Vitamin B	Vitamin C	Vitamin D	Fat	Cost	Attractiveness
1	18	4	-	6	-	10	6
2	8	6	22	14	-	6	1
3	22	10	-	-	8	8	5
4	6	12	-	6	-	16	5
5	-	14	-	10	-	8	6
6	-	-	20	8	6	3	4
7	8	-	-	6	20	6	10
8	4	-	8	-	40	8	10
9	-	6	10	12	-	12	6
10	12	20	-	4	-	12	7

	Minimum	Maximum
Vitamin A	100	-
Vitamin B	120	-
Vitamin C	200	-
Vitamin D	80	-
Fat	-	300

minimise

$$\sum_i c_i x_i$$

subject to

$$\begin{aligned} \sum_i v_i^A x_i &\geq 100 \\ \sum_i v_i^B x_i &\geq 120 \\ \sum_i v_i^C x_i &\geq 200 \\ \sum_i v_i^D x_i &\geq 80 \\ \sum_i f_i x_i &\leq 300 \\ x_i &\geq 0, \quad \forall i \end{aligned}$$

x_i - Number of food i (Variable)

v_i^A - Amount of vitamin A in food i (Constant)

v_i^B - Amount of vitamin B in food i (Constant)

v_i^C - Amount of vitamin C in food i (Constant)

v_i^D - Amount of vitamin D in food i (Constant)

f_i - Amount of fat in food i (Constant)

c_i - Cost of food i (Constant)

a_i - Attractiveness of food i (Constant)

5a Solution via Pyomo (Problem Based)

minimise

$$\sum_i c_i x_i$$

subject to

$$\sum_i v_i^A x_i \geq 100$$

$$\sum_i v_i^B x_i \geq 120$$

$$\sum_i v_i^C x_i \geq 200$$

$$\sum_i v_i^D x_i \geq 80$$

$$\sum_i f_i x_i \leq 300$$

$$x_i \geq 0, \quad \forall i$$

```
1 from pyomo.environ import *
2
3 # -----
4 # Data (simple Python lists)
5 # -----
6 cost = [10, 6, 8, 16, 8, 3, 6, 8, 12, 12]
7 vita = [18, 8, 22, 6, 0, 0, 8, 4, 0, 12]
8 vitb = [4, 6, 10, 12, 14, 0, 0, 0, 6, 20]
9 vitc = [0, 22, 0, 0, 0, 20, 0, 8, 10, 0]
10 vitd = [6, 14, 0, 6, 10, 8, 6, 0, 12, 4]
11 fat  = [0, 0, 8, 0, 0, 6, 20, 40, 0, 0]
12
13 n = len(cost) # number of foods
14 |
15 # -----
16 # Model
17 # -----
18 model = ConcreteModel()
19
20 # Decision variables: amount of each food
21 model.x = Var(range(n), domain=NonNegativeReals)
22
23 # Objective: minimize total cost
24 model.obj = Objective(expr=sum(cost[i] * model.x[i] for i in range(n)),
25                        sense=minimize)
26
27 # Vitamin and fat constraints
28 model.vita_min = Constraint(expr=sum(vita[i] * model.x[i] for i in range(n)) >= 100)
29 model.vitb_min = Constraint(expr=sum(vitb[i] * model.x[i] for i in range(n)) >= 120)
30 model.vitc_min = Constraint(expr=sum(vitc[i] * model.x[i] for i in range(n)) >= 200)
31 model.vitd_min = Constraint(expr=sum(vitd[i] * model.x[i] for i in range(n)) >= 80)
32 model.fat_max  = Constraint(expr=sum(fat[i] * model.x[i] for i in range(n)) <= 300)
```

Diet Problem (Tutorial Question 5b)

x_i - Number of food i

v_i^A - Amount of vitamin A in food i

v_i^B - Amount of vitamin B in food i

v_i^C - Amount of vitamin C in food i

v_i^D - Amount of vitamin D in food i

f_i - Amount of fat in food i

c_i - Cost of food i

a_i - Attractiveness of food i

Obtain the optimal solution given that the aim is to maximise the attractiveness of the diet.

minimise

$$\sum_i c_i x_i$$

subject to

$$\sum_i v_i^A x_i \geq 100$$

$$\sum_i v_i^B x_i \geq 120$$

$$\sum_i v_i^C x_i \geq 200$$

$$\sum_i v_i^D x_i \geq 80$$

$$\sum_i f_i x_i \leq 300$$

$$x_i \geq 0, \quad \forall i$$

maximise

$$\sum_i a_i x_i$$

subject to

$$\sum_i v_i^A x_i \geq 100$$

$$\sum_i v_i^B x_i \geq 120$$

$$\sum_i v_i^C x_i \geq 200$$

$$\sum_i v_i^D x_i \geq 80$$

$$\sum_i f_i x_i \leq 300$$

$$x_i \geq 0, \quad \forall i$$

Diet Problem (Tutorial Question 5)

Food	Vitamin A	Vitamin B	Vitamin C	Vitamin D	Fat	Cost	Attractiveness
1	18	4	-	6	-	10	6
2	8	6	22	14	-	6	1
3	22	10	-	-	8	8	5
4	6	12	-	6	-	16	5
5	-	14	-	10	-	8	6
6	-	-	20	8	6	3	4
7	8	-	-	6	20	6	10
8	4	-	8	-	40	8	10
9	-	6	10	12	-	12	6
10	12	20	-	4	-	12	7

	Minimum	Maximum
Vitamin A	100	-
Vitamin B	120	-
Vitamin C	200	-
Vitamin D	80	-
Fat	-	300

maximise

$$\sum_i a_i x_i$$

subject to

$$\sum_i v_i^A x_i \geq 100$$

$$\sum_i v_i^B x_i \geq 120$$

$$\sum_i v_i^C x_i \geq 200$$

$$\sum_i v_i^D x_i \geq 80$$

$$\sum_i f_i x_i \leq 300$$

$$x_i \geq 0, \quad \forall i$$

What is the issue here?

UNBOUNDED!!

The program is free to pick foods with no fat. As long as the vitamin constraints are satisfied, we are in the feasible region and the objective is unbounded.

Fix

Maximum and minimum restrictions on both fat and vitamin content, e.g.
maximum content = minimum content + 20%.

	Minimum	Maximum
Vitamin A	100	120
Vitamin B	120	144
Vitamin C	200	240
Vitamin D	80	96
Fat	250	300

maximise

$$\sum_i a_i x_i$$

subject to

$$\begin{aligned} \sum_i v_i^A x_i &\geq 100, & \sum_i v_i^A x_i &\leq 120 \\ \sum_i v_i^B x_i &\geq 120, & \sum_i v_i^B x_i &\leq 144 \\ \sum_i v_i^C x_i &\geq 200, & \sum_i v_i^C x_i &\leq 240 \\ \sum_i v_i^D x_i &\geq 80, & \sum_i v_i^D x_i &\leq 96 \\ \sum_i f_i x_i &\geq 250, & \sum_i f_i x_i &\leq 300 \\ x_i &\geq 0, & \forall i \end{aligned}$$

5c Solution via Pyomo (Problem Based)

maximise

$$\sum_i a_i x_i$$

subject to

$$\begin{aligned} \sum_i v_i^A x_i &\geq 100, & \sum_i v_i^A x_i &\leq 120 \\ \sum_i v_i^B x_i &\geq 120, & \sum_i v_i^B x_i &\leq 144 \\ \sum_i v_i^C x_i &\geq 200, & \sum_i v_i^C x_i &\leq 240 \\ \sum_i v_i^D x_i &\geq 80, & \sum_i v_i^D x_i &\leq 96 \\ \sum_i f_i x_i &\geq 250, & \sum_i f_i x_i &\leq 300 \\ x_i &\geq 0, & \forall i \end{aligned}$$

```

1 from pyomo.environ import *
2
3 # -----
4 # Data (simple Python lists)
5 # -----
6 att = [6, 1, 5, 5, 6, 4, 10, 10, 6, 7]
7 vita = [18, 8, 22, 6, 0, 0, 8, 4, 0, 12]
8 vitb = [4, 6, 10, 12, 14, 0, 0, 0, 6, 20]
9 vitc = [0, 22, 0, 0, 0, 20, 0, 8, 10, 0]
10 vitd = [6, 14, 0, 6, 10, 8, 6, 0, 12, 4]
11 fat = [0, 0, 8, 0, 0, 6, 20, 40, 0, 0]
12
13 n = len(att) # number of foods
14
15 # -----
16 # Model
17 # -----
18 model = ConcreteModel()
19
20 # Decision variables: amount of each food
21 model.x = Var(range(n), domain=NonNegativeReals)
22
23 # Objective: minimize total cost
24 model.obj = Objective(expr=sum(att[i] * model.x[i] for i in range(n)),
25                        sense=maximize)
26
27 # Vitamin and fat constraints
28 model.vita_min = Constraint(expr=sum(vita[i] * model.x[i] for i in range(n)) >= 100)
29 model.vitb_min = Constraint(expr=sum(vitb[i] * model.x[i] for i in range(n)) >= 120)
30 model.vitc_min = Constraint(expr=sum(vitc[i] * model.x[i] for i in range(n)) >= 200)
31 model.vitd_min = Constraint(expr=sum(vitd[i] * model.x[i] for i in range(n)) >= 80)
32 model.fat_max = Constraint(expr=sum(fat[i] * model.x[i] for i in range(n)) <= 300)
33
34 # Additional constraints
35 model.vita_max = Constraint(expr=sum(vita[i] * model.x[i] for i in range(n)) <= 100 * 1.2)
36 model.vitb_max = Constraint(expr=sum(vitb[i] * model.x[i] for i in range(n)) <= 120 * 1.2)
37 model.vitc_max = Constraint(expr=sum(vitc[i] * model.x[i] for i in range(n)) <= 200 * 1.2)
38 model.vitd_max = Constraint(expr=sum(vitd[i] * model.x[i] for i in range(n)) <= 80 * 1.2)
39 model.fat_min = Constraint(expr=sum(fat[i] * model.x[i] for i in range(n)) >= 300 / 1.2)

```

Brief Introduction to Mixed-Integer Linear Programming (MILP)

Manufacturing Problem (Tutorial Q4 Adapted)

A firm manufactures 3 products, X, Y and Z.

Each one is processed by a single machine, machine A, which has a capacity of 100 hours per week.

It takes 4.5 hours to make product X, 2 hours to make product Y and 3 hours to make product Z.

- The profit made (per unit) for X, Y and Z is £11.5, £5 and £8 respectively.
- The firm only has enough material a week to make 20 of product Z

How many X's, Y's and Z's should be made per week so that the largest profit is made?

Manufacturing Problem (Tutorial Q4 Adapted)

maximise $P = 11.5x + 5y + 8z$ (objective)

subject to:

$4.5x + 2y + 3z \leq 100$ (machine hours constraint)

$z \leq 20$ (product Z constraint)

and

$x, y, z \geq 0$ (non-negativity constraint)

Solution

$$(x, y, z)^* = (8.89, 0, 20)$$
$$P^* = 262.62$$

Manufacturing Problem (Tutorial Q4 Adapted)

What if anything that isn't fully manufactured has to be discarded or it would cost the company overtime?

Perhaps we should only allow the solution to be completed products (i.e. whole numbers).

It transpires that if you round the solution up, $(9,0,20)$, you need 100.5 hours (INFEASIBLE). If you round down, $(8,0,20)$, you get a profit of £252, with a time taken of 96 hours.

However, if you make $(8,2,20)$, and get a profit of £262, with a time taken of 100 hours

A ten pounds might not seem like much, but imagine we are dealing in thousands or millions of pounds!

KEY POINT: Rounding a solution does not necessarily give you the optimal integer solution.

Desmos Demonstration

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

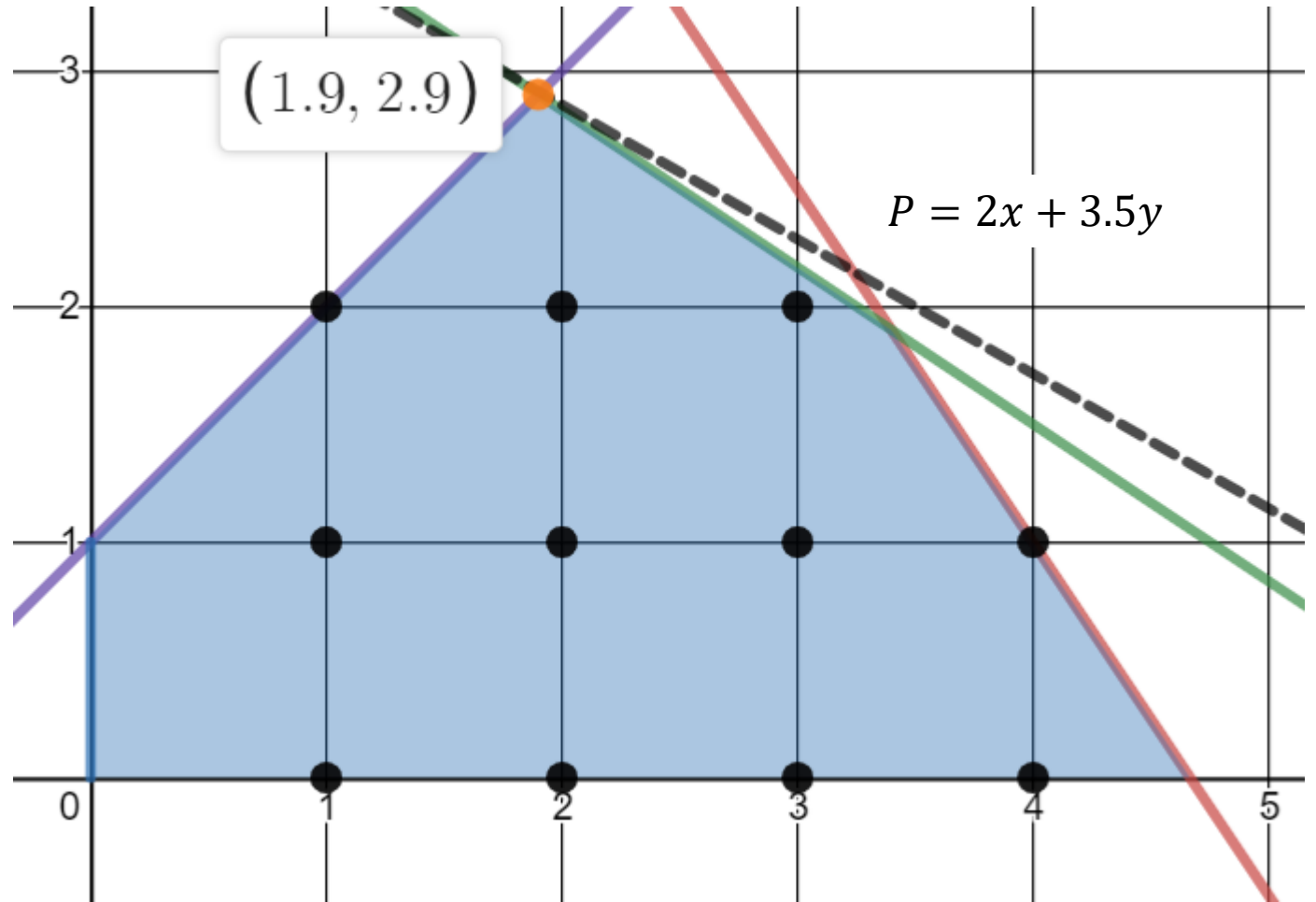
and

$$x, y \geq 0$$

Optimal Solution

$$x^* = (x, y) = (1.9, 2.9)$$

$$P^* = 13.95$$



Rounded Solutions

Optimal Solution

$$\mathbf{x}^* = (x, y) = (1.9, 2.9)$$
$$P^* = 13.95$$

$$\mathbf{x} = (x, y) = (1, 2)$$

$$P^* = 9$$

$$\mathbf{x} = (x, y) = (1, 3)$$

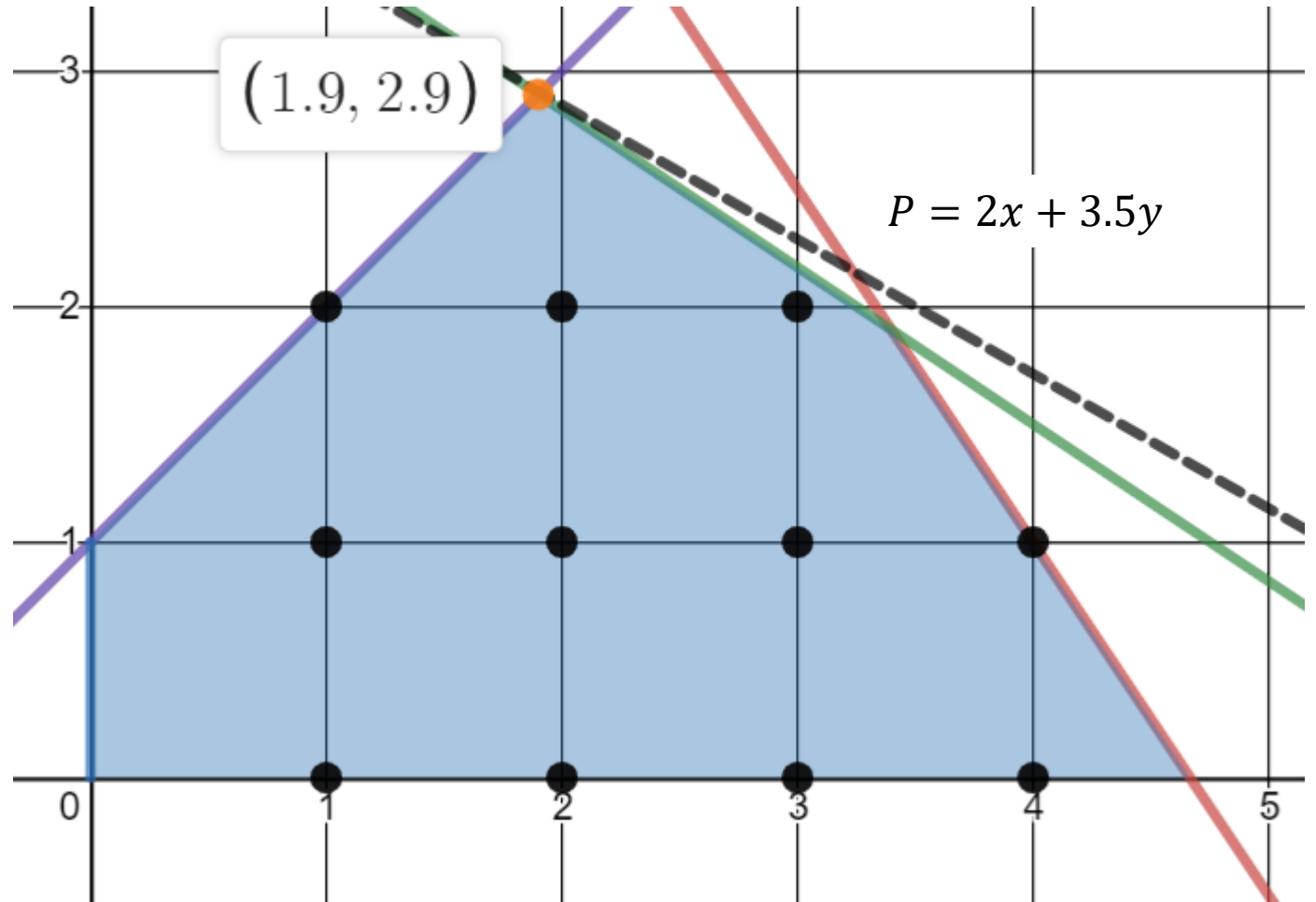
Infeasible!

$$\mathbf{x} = (x, y) = (2, 2)$$

$$P^* = 11$$

$$\mathbf{x} = (x, y) = (2, 3)$$

Infeasible!



Rounded Solutions

$$\mathbf{x} = (x, y) = (1, 2)$$

$$P = 9$$

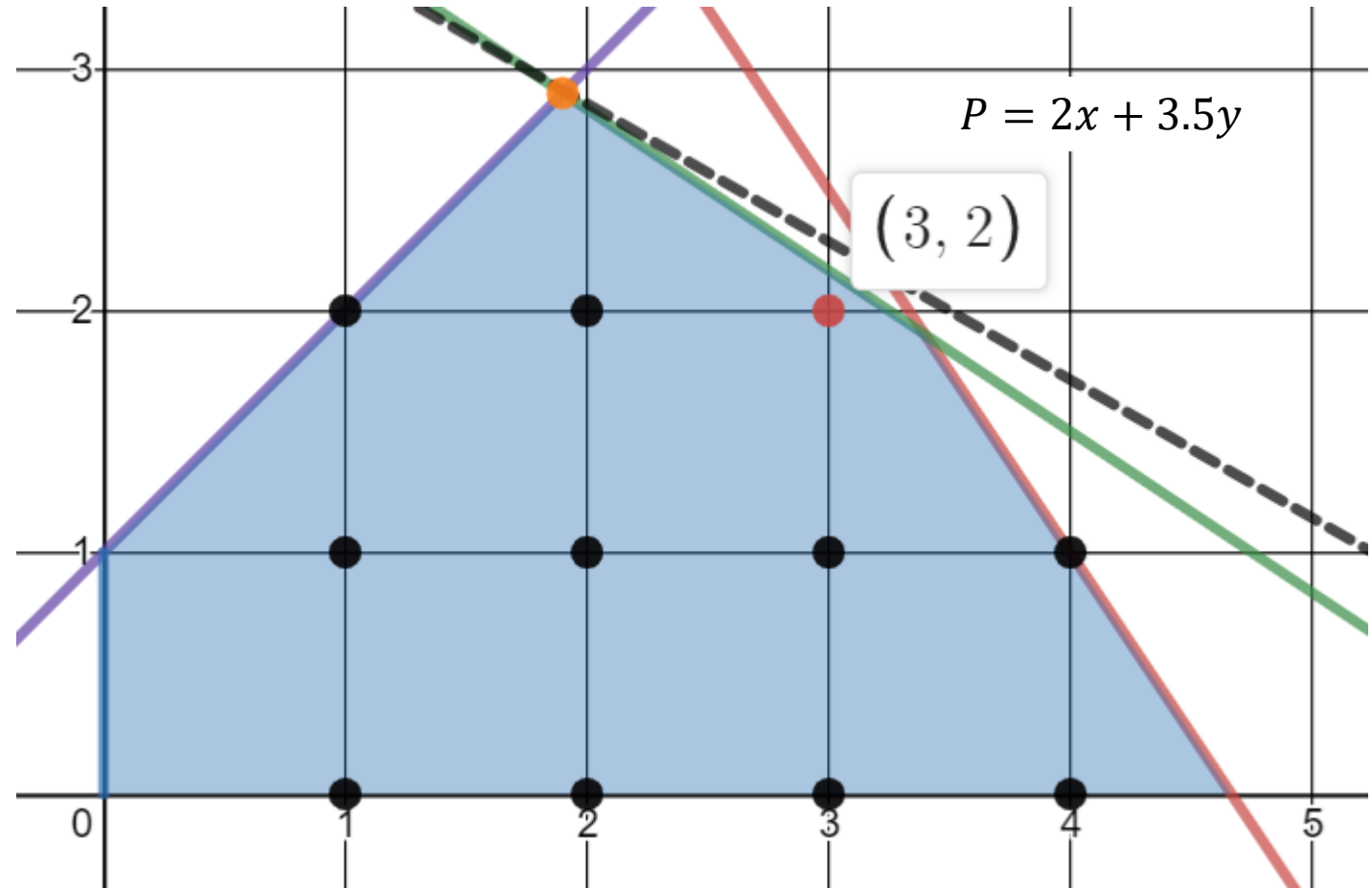
$$\mathbf{x} = (x, y) = (2, 2)$$

$$P = 11$$

However!

$$\mathbf{x}^* = (x, y) = (3, 2)$$

$$P^* = 13$$



Integer Solution via Pyomo (problem based)

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

and

$$x, y \geq 0$$

$$x, y \in \mathbb{Z}$$

Optimal Solution

$$x^* = (x, y) = (3, 2)$$

$$P^* = 13$$

Optimal solution found:
Model unknown

Variables:

x	:	Size=2,	Index=I				
	Key	Lower	Value	Upper	Fixed	Stale	Domain
0	:	0	:	3.0	:	None	: False : False : NonNegativeIntegers
1	:	0	:	2.0	:	None	: False : False : NonNegativeIntegers

Objectives:

obj	:	Size=1,	Index=None,	Active=True	
	Key	:	Active	:	Value
None	:	True	:	13.0	

```
1 from pyomo.environ import *
2
3 # Model
4 model = ConcreteModel()
5
6 # Index set and decision variables (both integer, >= 0)
7 model.I = Set(initialize=[0, 1])
8 model.x = Var(model.I, domain=NonNegativeIntegers)
9
10 # Objective: maximize 2*x[0] + 3.5*x[1]
11 f = {0: 2, 1: 3.5}
12 model.obj = Objective(expr=sum(f[i] * model.x[i] for i in model.I), sense=maximize)
13
14 # Constraints
15 model.c1 = Constraint(expr=3*model.x[0] + 2*model.x[1] <= 14)
16 model.c2 = Constraint(expr=2*model.x[0] + 3*model.x[1] <= 12.5)
17 model.c3 = Constraint(expr=-model.x[0] + model.x[1] <= 1)
18
19 # Show model formulation
20 print("Model formulation:")
21 model.pprint()
22
23 # Solve (GLPK supports MIPs)
24 solver = SolverFactory('glpk')
25 results = solver.solve(model, tee=False)
```

Mixed-Integer Program

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

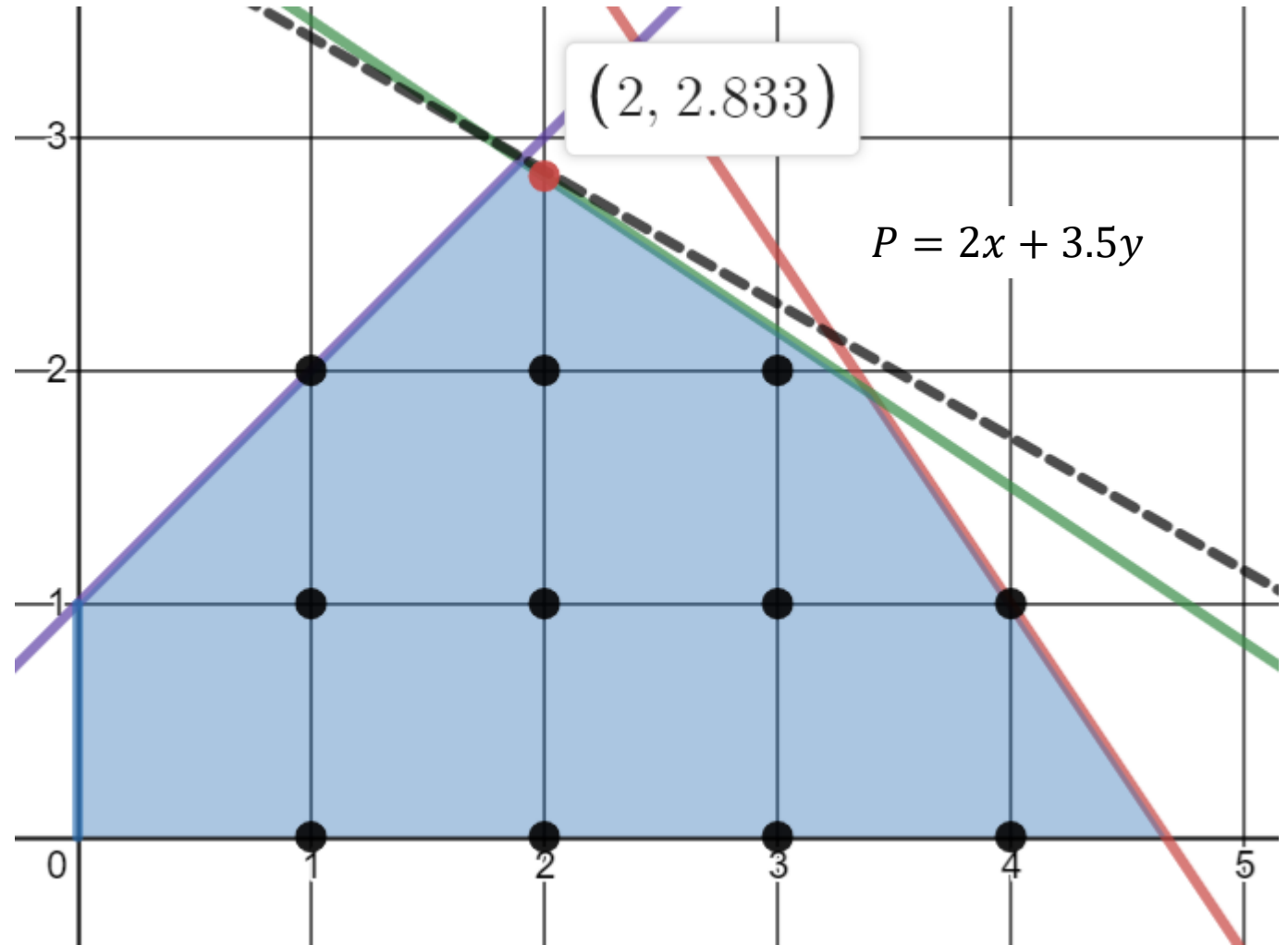
and

$$\begin{aligned} x, y &\geq 0 \\ x &\in \mathbb{Z} \end{aligned}$$

Optimal Solution

$$\mathbf{x}^* = (x, y) = (2, 2.833)$$

$$P^* = 13.917$$



Mixed-Integer Solution (problem based)

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$

$$2x + 3y \leq 12.5$$

$$-x + y \leq 1$$

and

$$x, y \geq 0$$

$$x \in \mathbb{Z}$$

Optimal Solution

$$x^* = (x, y) = (2, 2.833)$$

$$P^* = 13.917$$

Optimal solution found:
Model unknown

Variables:

```
x1 : Size=1, Index=None
     Key : Lower : Value : Upper : Fixed : Stale : Domain
         None : 0 : 2.0 : None : False : False : NonNegativeIntegers
x2 : Size=1, Index=None
     Key : Lower : Value : Upper : Fixed : Stale : Domain
         None : 0 : 2.833333333333333 : None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
     Key : Active : Value
         None : True : 13.916666666666655
```

```
1 from pyomo.environ import *
2
3 # Create a concrete model
4 model = ConcreteModel()
5
6 # Decision variables
7 model.x1 = Var(domain=NonNegativeIntegers, bounds=(0, float('inf')))
8 model.x2 = Var(domain=NonNegativeReals, bounds=(0, float('inf')))
9
10 # Objective function (maximize 2x1 + 3.5x2)
11 f = [2, 3.5]
12 model.obj = Objective(expr=f[0]*model.x1 + f[1]*model.x2, sense=maximize)
13
14 # Constraints
15 model.c1 = Constraint(expr=3*model.x1 + 2*model.x2 <= 14)
16 model.c2 = Constraint(expr=2*model.x1 + 3*model.x2 <= 12.5)
17 model.c3 = Constraint(expr=-model.x1 + model.x2 <= 1)
18
19 # Print model before solving
20 print("Model formulation:")
21 model.pprint()
22
23 # Solve with GLPK
24 solver = SolverFactory('glpk')
25 results = solver.solve(model, tee=False)
```

Demos

<https://colab.research.google.com/drive/1y20IIA3po-hK4UWxqkc-Hxk0h6babXIT?usp=sharing>